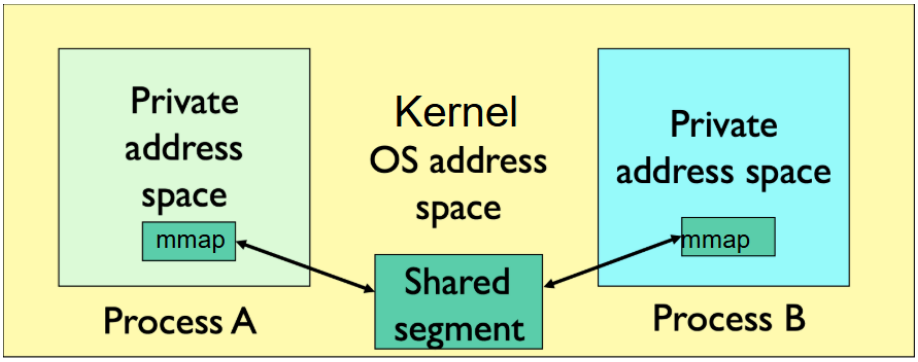
IPC

- inter process: between two processes
- involves data exchange between the processes; files, pipes, sockets, and signals
    - files are the slowest since on disk, requires locks so that Process A and B can't write to it at same time, also need to wait for Cmd1 to do before Cmd2
    - pipes: one directional data stream; between parent and child processes
    - sockets: bidirectional, can be within computer and through the internet
- job: 2 or more executable commands and their pipe(s); 2 or 3 more processes
- User Mode and Kernel Mode: controlled by the kernel itself
    - kernel threads: don't interact with users and remain alive until system is shut down
    - **reentrant**: several processes can be in kernel mode at the same time; can be interrupted and then called again by another execution; usage of local variables and more.
    - kernel control path: think like a kernel function, must be async
    - race condition: happens if outcome of something depends on the order of processes that are scheduled
    - interrupt: hardware signal; asynchronous; electrical signal for cpu to stop and switch to kernel mode
    - exception: synchronous; when a cpu attempts to do something that is problematic (div by 0, page faults); switches to kernel mode which fixes it
    - system call: from a user process, intentional exception that is programmed, a trap to pause itself and let OS perform a privileged action.
- **mmap()**: syscall pointing disk data to spaces in memory; when accessed, lazy loaded by kernel
    - i/o treated like memory access, use MAP_SHARED so can share same memory mapped area
    - #include <sys/mman.h>
    - void *mmap(void *addr, size_t len, int prot, int flag, int fd, off_t off);
    - Returns: starting address of mapped region if OK, MAP_FAILED on error
    - addr: it is just a variable of a mem address. the starting memory address into which to map the file (NULL means not previously allocated; argument can just be NULL)
    - len: the length of the data to map into memory
    - prot: the kind of access to the memory mapped region (read/write/exec)
    - flags: flags that can be set for the system call:
        - **map_shared** -processes mapping same file region can view updates from other processes
        - **map_private** -updates to the mapping are not visible to other processes mapping same file
    - fd: file descriptor
    - off: the offset in the file to start mapping from

-

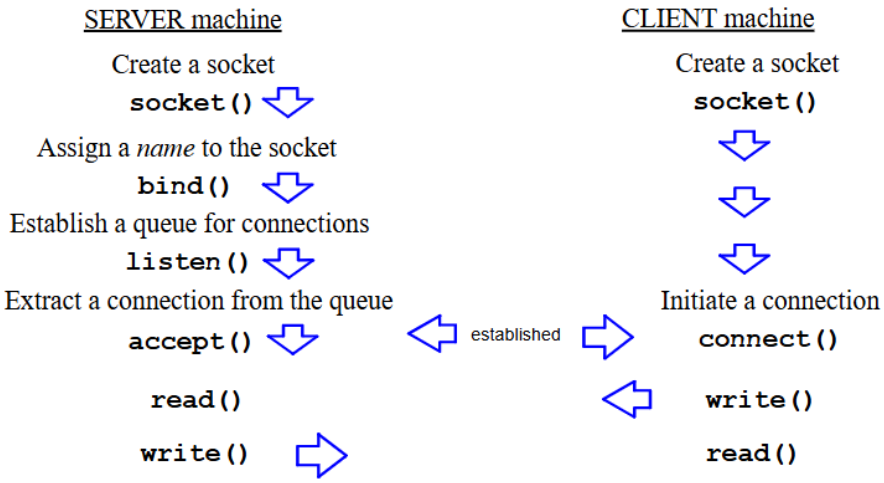


mmap() establishes a mapping of a named file system object (or part of one) into a process address space. It is the basic memory management function and it is very simple.

- First open() the file, then
- mmap() it with appropriate access and sharing options
- Away you go.
  - **SET** MAP_SHARED || MAP_ANONYMOUS , FD=-1 to enable Inter-Process Communication (IPC), then two communicating programs access the shared memory, creates a semaphore and ensures exclusive access to shared memory
  - multi-directional, more than 2 processes; much more memory than pipe buffers and syncs with file system with msync()
  - int msync(void *addr, size_t len, int flags);
  - int munmap(void *addr, size_t len);

Socket Connection Paradigm

Network programming/protocols use sockets (FTP, SMTP, POP3, WWW)



- synchronization: want all processes to be synchronized, not fight over resources among other things, basically protect shared resources from being accessed by more than one process at time.
    - preemption disabling: process executed in kernel mode cannot be interrupted by another process tryna do some sys call; internal variable that basically says to not switch
    - interrupt disabling: don't listen to hardware interrupts; only for single processors
    - **semaphores**: counter with some data structure (queue)
        - int variable: current state of the semaphore
        - list of waiting processes (queue)
        - down(); decrease state, if < 0 add process to list and invoke scheduler (switch to diff process)
        - up(); increase, if > 0 pop one or more processes and run them from the list
    - **spin lock:** spins a resource until unlocked; semaphore without the lsit; only for mutliprocessors
    - **deadlock:** process1 gets access to a and process2 gets access to b; p2 then waits for a and p1 waits for b. since already accessed, endless loop
- signals: notifying processes of system events, have a number
    - asynchronous: sigint, when user interrupts foreground process, not necessarily waiting for it to happen and can happen at any moment
    - synchronous: when waiting memory, kernel sends signal when at invalid, waiting for some return value
    - 2 actions: ignore, or asynchronously execute a sighandler
    - default actions if not specified:
        - terminate process
        - write contents to a file in addr space (core dump)
        - ignore the signal
        - suspend the process
        - resume the process's execution
- system v ipc: interprocess communication other than signals: semamphores, message queues, and shared memory
    - shmget(), semget(), msgget()
    - msgsnd() and sgrcv() calls to insert messages into specific queues
    - shmget() fastest (shared memory), shmat() to attach memory, shmdt() to detach memory
- **PIPES; ONE WAY!**
    - int pipe(int filedesc[2]); basically pass in an array length 2 holding the file descriptors of the buffer, one (filedesc[0]) for reading and one (filedesc[1]) for writing, data sent in a queue
    - multiple children: lets say you pipe from parent to child, then child forks. all the children copy this and the file descriptor table; two readers and two writers
    - must close these connections to the pipe with close(filedescriptor)

- `fcntl()` (file control) system call with the `F_SETPIPE_SZ` command, a user process can explicitly dictate the exact byte size of the pipe's underlying circular buffer.

```
int main(void) {
    int pfds[2];
    char buf[30];
    pipe(pfds);
    if (!fork()) {
        printf(" CHILD: writing to pipe\n");
        write(pfds[1], "test", 5);
        printf(" CHILD: exiting\n");
    } else {
        printf("PARENT: reading from pipe\n");
        read(pfds[0], buf, 5);
        printf("PARENT: read \"%s\"\n" , buf);
        wait(NULL);
    }
    return 0;
}
```

Available under code/ipc/parent_child_pipe.

- unidirectional! close all the stuff ur not gonna use for the pipe
- close sends EOF
- use mkfifo to make and name your own pipe; bidirectional!
  cs149@cs149-VirtualBox:~$ mkfifo my_pipe
- cs149@cs149-VirtualBox:~$ gzip -9 -c < my_pipe > out.gz &
- [1] 3278
- cs149@cs149-VirtualBox:~$ cat myfile > my_pipe
- cs149@cs149-VirtualBox:~$ ls -altr out.gz
- -rw-rw-r-- 1 cs149 cs149 622 Sep 21 12:18 out.gz

signals

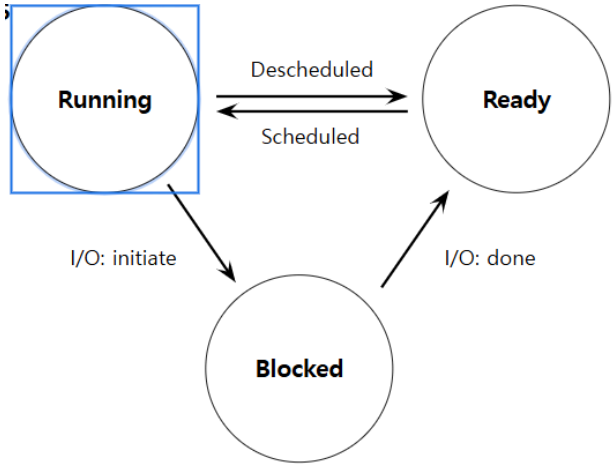
- event notifications from user, hardware, or other processes; handled and mediated by the kernel
- signal handlers: per process, if signal happens, do this to the process
- Ansi C and POsix
- SIGKILL/SIGTERM kill command
- SIGINT The process was “interrupted”. This happens when you press Control+C on the controlling terminal.
- SIGHUP If a process is being run from terminal and that terminal suddenly goes away then the process receives this signal. “HUP” is short for “hang up” and refers to hanging up the telephone in the days of telephone modems
- SIGCHLD The process had previously created one or more child processes with the fork() function. One or more of these processes has since died.
- SIGSEGV An attempt was made to access memory not allocated to the process.

This is often caused by reading off the end of arrays etc

- **Signals (software interrupts)**: kernel-mediated notification of an event that is sent from the OS kernel to a specific process - there can be many events causing a signal!
    - some event happens in a process, which forces kernel to step in
    - kernel steps in, yields what is happening, and sends a signal
    - the default or user made sighandler will handle this and stuff.
    - can stop or continue to application afterwards
    - main way of communicatino from kernel to a process

```
$ trap -l (or kill -l)
 1) SIGHUP   2) SIGINT   3) SIGQUIT  4) SIGILL
 5) SIGTRAP  6) SIGABRT  7) SIGEMT   8) SIGFPE
 9) SIGKILL 10) SIGBUS  11) SIGSEGV 12) SIGSYS
13) SIGPIPE 14) SIGALRM 15) SIGTERM 16) SIGURG
17) SIGSTOP 18) SIGTSTP 19) SIGCONT 20) SIGCHLD
21) SIGTTIN 22) SIGTTOU 23) SIGIO   24) SIGXCPU
25) SIGXFSZ 26) SIGVTALRM  27) SIGPROF 28) SIGWINCH
29) SIGINFO 30) SIGUSR1 31) SIGUSR2
```

    -
    - #include <signal.h>; using the Posix, not ANSI C notation
    - SIGINT: interrupt, SIGSEGV: segemntation fault
    - SIGIGN: install to ignore signals, SIGDFL to restore default signal handler
    - Every process has a "pending signal" memory area where this value is stored
    - stored in a data structure, call do_signal() that either kills, does the default, or does the user defined handler (handle_signal)
    - applications can define own signals using unused values (remember, each signal is mapped to an integer)
    - Sigusr1 and sigusr2 don't have handlers adn you need to define handlers for them, can't change handlers for sigkill/sigstp
    - ***kill -signal pid*** ***killall -signal processname***
        - default siganl is sigterm
    - fg command brings process to foreground (unpause); uses JOB IDs, not PIDs
    - **sigaction()** **int sigaction(int iSig, const struct sigaction *psAction**
    - **struct sigaction sAction;**
    - **sAction.sa_flags = 0;**
    - **sAction.sa_handler = myHandler;**
    - **sigemptyset(&sAction.sa_mask);**
    - **iRet = sigaction(SIGINT, &sAction, NULL);**
    - **assert(iRet == 0);**
    - **#define _GNU_SOURCE**
    - **#include <stdio.h>**
    - **#include <stdlib.h>**
    - **#include <signal.h>**
    -
- **Trap**: any event that causes a switch to kernel mode, such as for a system call

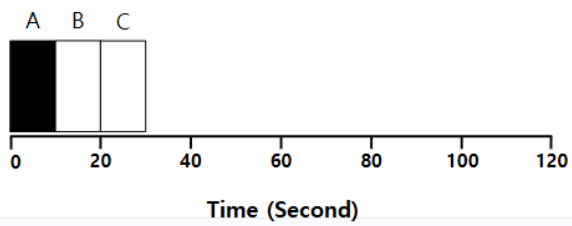
    - kill() syscall to send a signal to particular process
    - raise() ANSI C format, obsolete
- **Exception**: any unexpected event generated from within the CPU, such as divide by zero (turns into a SIGFPE)
- **Hardware Interrupts**: a hardware-generated event from outside the CPU, such as typing Ctrl-C on a keyboard
    - mediated by hardware, handled by the kernel (can be handled by throwing a signal)
    - Type Ctrl-C: sigint (2), exit the process gracefully ; can be caught/ignored, happy version
    - Type Ctrl-Z sigtstp (20), suspend process, do sigcont (18) to unsuspend
    - Type Ctrl-/ sigquit (3), exit and core dump gracefully, kills children; can be caught as well, unhappy
    - Ctrl-D is EOF!
    - sigterm: kill process, can be caught/ignored, not graceful but can still cleanup
    - sigkill: kill process NOW! last resort
- processes are assigned groups (pgid), known as jobs; hwo to distribute signal to all
- 

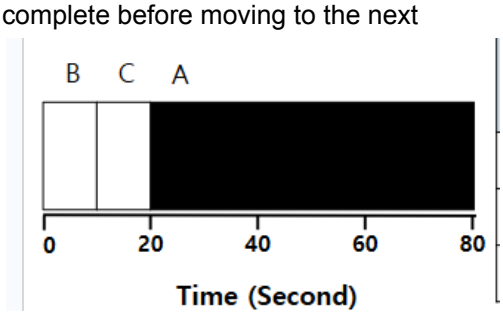
scheduling

- process states
    - running: process currently on the processor and running
    - ready: ready to run, but OS decides not your time yet, ready to be scheduled to run
    - blocked: did some i/o request adn blocked -> lets another processor use it
- process control blocks: basically structures that store information about the process, like the registers at the time of the running process, like execution state as well as the id, status, priority
- context switch, save current process in process control block and find another one that is ready
- `PROCESS STATE CODES`
- `Here are the different values that the` **`s, stat`** `and` **`state`** `output specifiers (header "STAT" or "S") will display to describe the state of a process:`
- `D  uninterruptible sleep (usually IO)`

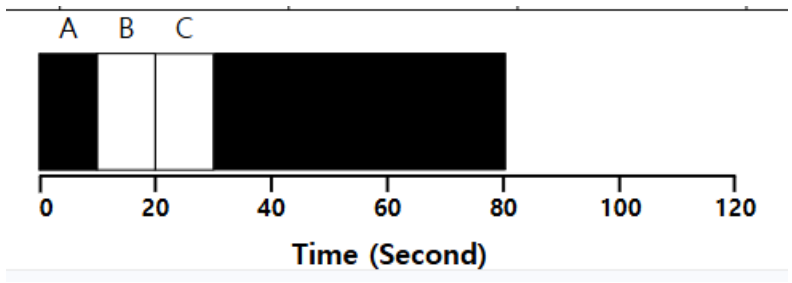
- `I  Idle kernel thread`
- `R  running or runnable (on run queue)`
- `S  interruptible sleep (waiting for an event to complete)`
- `T  stopped by job control signal`
- `t  stopped by debugger during the tracing`
- `W  paging (not valid since the 2.6.xx kernel)`
- `X  dead (should never be seen)`
- `Z  defunct ("zombie") process, terminated but not reaped by its parent`
- These are indeed the process states. Processes states that ps indicates are:
- I Marks a process that is idle (sleeping for longer than about 20 seconds).
- R Marks a runnable process.
- S Marks a process that is sleeping for less than about 20 seconds.
- T Marks a stopped/suspended process.
- U Marks a process in uninterruptible wait.
    - process that happens to be in a system call that cannot be affected by ANY signals
    - usually waiting for specific hardware like NFS, or other tsuff amongh other things
- Z Marks a dead process (a ``zombie'').
- D Uninterruptible sleep (usually IO) R Running or runnable (on run queue) S Interruptible sleep (waiting for an event to complete) T Stopped, either by a job control signal or because it is being traced. W paging (not valid since the 2.6.xx kernel) X dead (should never be seen) Z Defunct ("zombie") process, terminated but not reaped by its parent.
- and the additional characters are:
- < high-priority (not nice to other users)
- N low-priority (nice to other users)
- L has pages locked into memory (for real-time and custom IO)
- s is a session leader
- l is multi-threaded (using CLONE_THREAD, like NPTL pthreads do)
- + is in the foreground process group

- prioritizing the faster processes to the slower processes
- scheduler runs on process id of 0; decides which ready job to run
- workload: job descriptions like arrival_time, start_time, and duration_time
- preemption: temporarily interrupt a task and continue later (involves context switching)
- arrival_time: when the process arrives on the ready queue ready to be ran
- completion_time: when process is exited/terminated
- start_time: when the process starts running
- want to minimize turnaround time, completion_time - arrival_time
- also minimize response time; start_time - arrival_time
- “10% tax for low income person feels worse than 10% for wealthy, thus higher income brackets”
    - Behavior of a process may be very interactive (user mouse clicks) or i/o bound but you dont know that from the start.
    - You also dont know in reality how long processes will run for
    - Performance and fairness are often at odds in scheduling
- optimal workload, where scheduler is just a queue (FIFO)

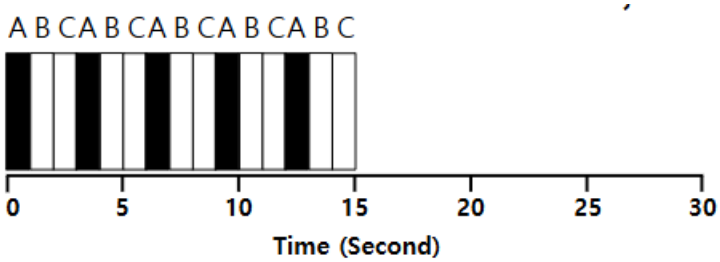
    - each job runs for same amount of time
    - all jobs arrive at the same time, meaning that from the queue to the cpu it takes the same amount of time
    - each job's runtime is known
    - all the jobs uses ONLY the CPU with NO I/O
- schedulers
    - FIFO (queue, or FCFS first come first served)
        - all the naive assumptions
        - jobs are run in arrival time order (basically in what order they were added to the queue, first in and out
        - good for lists of jobs
            - A is before B is before C on a list/queue.
            - Each job runs for 10 seconds.
        - 

        - really slow, C's turnaround time depends on wahtever is ahead of it adn their speeds, so for this example it is 30 seconds since it has to wait for A and B to finish then run ;short jobs have to wait for the long jobs in front of it
        - average = add up the turnarounds / number of items
    - SJF (shortest jobs first)
        - no longer assume all jobs are the same length
        - optimize the tractor in front of the cars, slowing everyone down
        - non-preemptive scheduler, doesn't interrupt tasks at all and waits until they fully complete before moving to the next
        - 

        - only works if you know all jobs arrive at the same time and you know how long they will run for, it will be optimal; think like this is static like you are handed the list and it doesn't update
    - STCF(shorteset time to completion first)
        - no longer assume jobs are same length and arrive at the same time
        - optimize when new jobs are added and is dynamic compared to the static stuff
        - same as above, but the preemptive version, so when a new job enters, find the projected duration and schedule the next job to whatever has the least time left



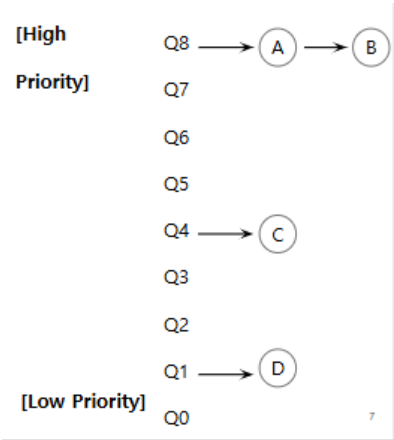
- RR(Round Robin)
    - now, we stop assuming each job runs for same amount of time, jobs arrive at the same time, and we know run time of job is known
    - optimizes response time whereas the previous didn't so if you need interactibility this is pretty decent; shit at turnaround time tho
    - run a job for a time slice (aka scheduling quantum, its an OS Constant) then switch to next job until the queue is finished
    - time slice must be a multiple of the timer-interrupt period (there is a clock that tells the CPU when to switch, so it basically switches to new one based on this clock)



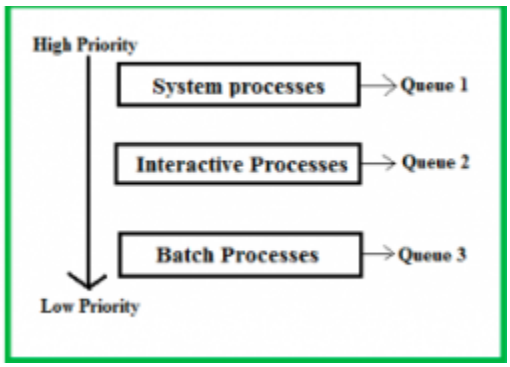
    - longer the time slice, less context switching but worse response time (vice versa also is true); in the end what is the priority, do you want to optimize context switching or how interactive it feels?
- Incorporating I/O
    - don't want to have processes that wait for I/O to use up CPU
    - preempt blocking with other processes, making up for the unused time
- CPU Utilization is what portion of the clock cycles the CPU is actually doing computations vs sitting idle for I/O to finish
- Throughput is the number of processes that are completed per time unit

Multi Level Feedback Queue

- how the cpu schedules stuff, it actually has multiple queues with different priorities (think like fast lane, middle lane, and a slow lane)
- MLFQ basically observes the execution of each job and learn its attributes, prioritizing it accordingly; lets it be good at both
    - based on a 1962 paper which is so cool



- distinct queues
- queue contains ready to run jobs in the queue with the highest priority
- the scheduler will choose from these queues to run the jobs with the highest priority
- if multiple jobs are on queue, then you either run FIFO or RR
- put less nice jobs in higher priority, meaning they don't need as much cpu utilization and stuff; do RR on them (since they will be preempted anyway best to put up here)
- put nicer jobs in lower prioerity since are CPU intesnive adn take long to run, do FIFO on them

- There's one special scenario in which a process is interrupted by a hardware interrupt (e.g. a clock tick) interrupts user mode executing code, and a pending signal is detected. Then, before returning back to user code, a special code sequence is needed and installed in the user process' stack, to execute the signal handler and issue interrupt-return code (this meaning a special machine code sequence to execute the signal handler and then restore completely the cpu status before the system returns to what the user process was doing before the clock tick) in order to cope with the case that the interrupt came in the middle of some sensible task (an asynchronous interrupt can come at any point in time, between any user code machine instructions)
- 

- basically what we will prioritize. since we can't assume too miuch of the current thing, we have to both optimize response time as well as turnaround time!
    - GUI, I/o bound; notices it keeps relinquishing cpu to wait for I/O. since it keeps preempting we put it as high priority
    - long-running, cpu-bound; reduce its priority and do like LIFO on it.
- Rules:

    - if Priority(A) > Priority(B), A runs (B doesn't)
    - if Priority(A) = Priority(B), A and B run in RR or FIFO
    - When a job enters the system, it is placed at the highest priority
    - If a job uses up an entire time slice while running (behvaes like it is CPU-bound), its priority is reduced
    - If a job gives up the CPU before the time slice is up, it stays at the same priority level
    - after some time period S, move all jobs in the system to the topmost queue
    -

- allows for processes to be classified and change groups when need be
- Problems
    - starvation: i/o takes all the cpu, longer jobs don't get cpu time due to the I/O
        - fix: prioirty boost: after some time period S, move all the jobs to the top most queue and restart the MLFQ; rule 5
    - game the scheduler: 99% of time slice used, never gets priority reduced and those monopolizes CPU time
        - fix: once a job uses up its time allotment at a given level, priority is reduced, means we need to define how much time total can be used and then reduce its priority; rule 4a,b
    - a process may chnage its behavior over time
        - fix: tune the fuck out of the scheduler
            - high priority: <10ms for each time slice
            - low prioirty: ~100ms for each time slice
- other types of schedulign:
    - fair-share: everyone gets a fair share of cpu through some lottery system
    - mutliprocessing: which cpu for which job?
- nicencess: priority of a job, ranges from -20(high priority) to 19(low priority), where default is 0.
    - add 20 to derive actual priority
    - higher priority, less nice, lower priority, more nice.
    - how nice a process is to the other process, will it be greedy and take all the stuff or not?
    - nice command: `nice -n 5 ls -lart`
        - no value, default is 10; only root can run this to have a higher priority
    - renice command if it is already running
    - if same nice value, parent and child split 50 50. once child gets a lower priority, parent gets 98.5 while child gets 1.5

- **RLIMIT_NICE** - The limit to which a process's nice value can be raised to affect its scheduling priority
- **RLIMIT_DATA** - The maximum amount of memory that a program can allocate for its data. Additional allocation beyond this limit will fail.
- **RLIMIT_NPROC** - The maximum number of child processes that can be running for this user. If the process calls fork and too many processes belonging to this user are running on the system, fork fails.
- **RLIMIT_NOFILE** - The maximum number of file descriptors that the process may have open at one time.
- **RLIMIT_CPU** - Limit in secs on the amount of CPU time that a process can consume.

```
struct rlimit rl;

/* Obtain the current limits. */
getrlimit (RLIMIT_CPU, &rl);
/* Set a CPU limit of 1 second. */
rl.rlim_cur = 1;
setrlimit (RLIMIT_CPU, &rl);
```

40

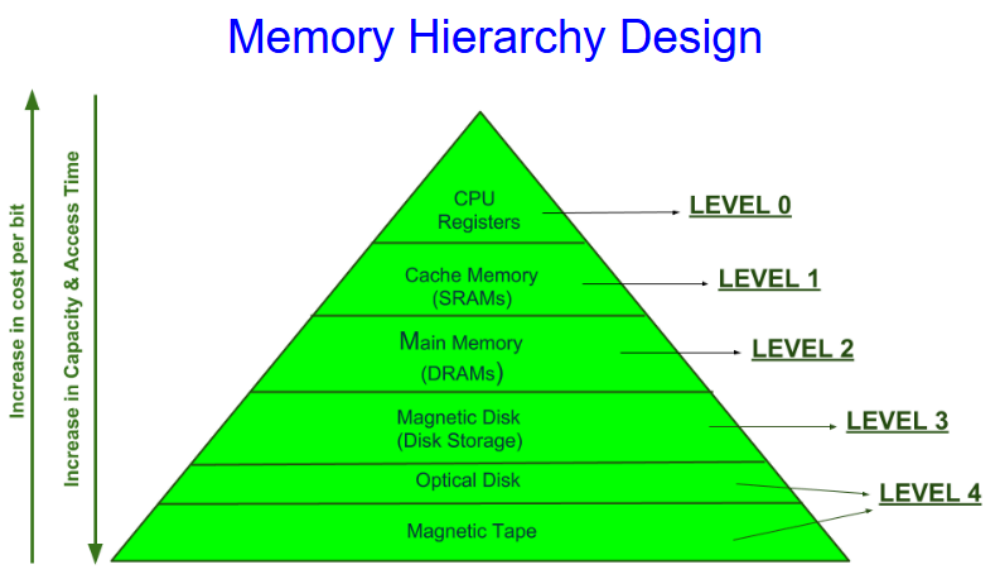
- use time command to time stuff

```
User CPU time is the time spent executing code in user mode.
System CPU time is the time spent by the kernel executing in
system mode on behalf of the process (e.g., executing system calls).
```

- wall clock (real) time - ms elapsed from now to start of process
- user CPU time (user mode) - ms in user mode executing user code
- system CPU time (kernel mode) - ms in kernel mode executing sys calls

Address space

- x86: family of instruction set architectures initially developed by intel; amd and intel still produce processors that follow this instruction set; nearly fully backwards compatible



- registers:

    - eax: accumulator, return value
    - ecx: loop counter
    - esi, edi: index registers, values of parameters of function
    - esp: stack pointer, used for like adding/removing stuff from stack
    - ebp: base pointer, where the stack starts basically for that function
    - segment: registers that hold entries to the GDT (kernel lookup table) of where the code, data, or stack segment starts
        - cs: code segment
        - ds: data segment
        - ss: stack segment
    - flags:
        - Carry, Overflow, Sign, Zero flag
    - eip or pc: address of the next instruction
    - ir: holds instruction currently being execute/decoded
- binary addition: pretty much like a truth table
    - XORing adds without carry
- hex addition: just add them hoes up and find the carry and stuff. no need to change to binary if u alr know all of em
    - A = 10, B = 11, C = 12, D = 13, E = 14, F = 15
- replacement policy dictates if a cache line is replaced or not
- primary storage vs secondary storage
    - primary: volatile (ony memory when power), internal (ram, rom, cache), fast access, temp memory, small amount of memory and expensive
    - secondary: non volatile, external (HDD, CD , DVD, flash, tape, SSD), cpu doesn't directly access, slow, permanent memory; larger sizes for bulk writes and much cheaper
- OS virtualizes physical memory, so each process looks like they can use all of the memory
- segmentation fault: when program tries to read/write to an illegal memory location
- program memory:
    - text segment: program instructions (like the actual code)
    - data segment: defined in compile time (global/static variables0
    - stack segment for temporary/automatically typed variables in subroutines/functions (local variables, pointers)
    - heap segment for objects allocated during runtime by functions
- benefits of virtualization:
    - time sharing, multiprogramming; looks like all of the processes running at the same time
    - isolation from other processes 0xA isn't gonna affect 0xA in process B
    - pointers won't be too bad, as relative to the process and not the global
    - done through the MMU that translates virtual to physical address
- heap: dynamically allocate memory during runtime

- stack: return addresses/values, local variables
- these are in the user-space of virtual memory, but there is also kernel space for kernel code (system calls, done through software interrupt using 0x80) and the global virtual table.
- malloc(): allocate on heap; returns a null pointer which you have to type
    - calloc is initialize with like a 0 and not just null
    - uses sbrk()
- free(ptr); free the malloc pointer, must be done
- `int *p = (int *) malloc(sizeof(int)); //on the heap`
- `p = NULL; //burns the bridge to malloc'ed memory`
- `free(p);`
    - this leads to a memory leak: pointer p to null, and what happens to malloced memory is it just stays there since p now points to null
- top command: sort processes by memory usage
- /proc/meminfo shows system memory usage
- pmap shows virtual memory map of a process

memory api

- you can actually write your own memory allocator if you don't like malloc
    - tcmalloc, jemalloc

malloc()

```
#include <stdlib.h>

void* malloc(size_t size)
```

- Allocate a memory region on the heap.
    - Argument
        - `size_t size` : size of the memory block (in bytes)
        - `size_t` is an unsigned integer type.
    - Return
        - Success : a void type pointer to the memory block (in the virtual address space) allocated by `malloc`
        - Fail : a NULL ('\0' or 0) pointer

free()

```
#include <stdlib.h>

void free(void* ptr)
```

- Free a memory region that was previously allocated by a call to `malloc`.
  - Argument
    - `void *ptr` : a pointer to a memory block that was allocated with `malloc`
  - Return
    - none

Other memory APIs: calloc()

```
#include <stdlib.h>

void *calloc(size_t num, size_t size)
```

- Allocates memory on the heap and ***zeroes it*** before returning.
  - Argument
    - `size_t num` : number of blocks to allocate
    - `size_t size` : size of each block (in bytes)
  - Return
    - Success : a void type pointer to the memory block allocated by `calloc`
    - Fail : a NULL pointer (0, which is reserved, i.e. you cannot have memory allocated at address 0)

Other memory APIs: realloc

```
#include <stdlib.h>

void *realloc(void *ptr, size_t size)
```

- Change the size of memory block previously allocated with malloc.
  - The pointer returned by `realloc` may be the same as `ptr` or a new pointer.
  - Argument
    - `void *ptr`: Pointer to memory block allocated with `malloc/calloc/realloc`
    - `size_t size`: New size for the memory block (in bytes)
  - Return
    - Success: (void *) type pointer to the memory block
    - Fail : NULL (0) pointer

- after free, put the pointer to null
- initialize a pointer before using it! (using malloc)
  - *ptr = n sets the value of what the pointer is pointing to to n

    - ptr = n sets the mem address to n (carefully, you can't dealloc the thing it was pointing to earlier!
- can access any locally scoped variable through pointer with & (address)
    - int myvariable = 5; //on the stack
    - int *ptr = & myvariable;
- char[] vs char *

```
 char a[] = "hello";  // array
//a string literal has a fixed size and C computes size 6
//because it is NULL-terminated. This a string is on the stack
   +---+---+---+---+---+---+
a: | h | e | l | l | o |\0 |
   +---+---+---+---+---+---+


 char *p = "world"; // pointer
// string literal "world" (constant) is allocated on the Data Segment,
// it is NULL-terminated, and p points to it. Cannot modify it
   +-----+     +---+---+---+---+---+---+
p: |  *======> | w | o | r | l | d |\0 |
   +-----+     +---+---+---+---+---+---+
```

    -
    - stored on stack vs stored on heap
    - can use char[] in place of char * in functional arguments
    - char[] can also be modified whereas char *p can't; can be cast to char *p but not the other way around
    - char * need to malloc or above; char[] don't need to start wiith size and also partially initialized will be padded with 0s
    - use sizeof() for malloc sizing
- potential issues:
    - forgetting to allocate memory, lead to segfault (malloc for dst)

```
char *src = "hello";
char *dst;          //
strcpy(dst, src);
```

    - not allocating enough memory; remember strings always end with '\0' character and take into account (just do strlen(src) + 1)

```
char *src = "hello"; //character string
char *dst = (char *)malloc(strlen(src));
strcpy(dst, src);    //doesn't work prop
```

    - forgetting to initialize a value; use calloc for default

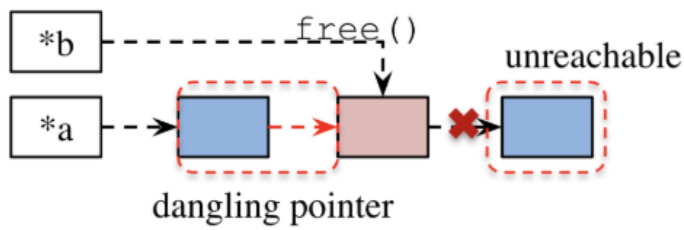
```
int *x = (int *) malloc(sizeof(int));
printf("*x = %d\n", *x); // uninitiali
```

    - memory leaks, where you allocate memory but then you don't free it afterwards and it remains unused -> running out of memory and dying

```c
#include<stdio.h>

int main()

{

   int *a = (int *)malloc(sizeof(int));

   int *b = (int *)malloc(sizeof(int));

  a = b; //Lose a

   b = NULL; //Lose b

   int *c = (int *)malloc(sizeof(int));

   int *d = (int *)malloc(sizeof(int));

   d = c; //Lose d

   c = NULL; //Lose c
```

29

- dangling pointer, free memory prematurely when still using the memory; opposite of leak; think like you alr called free, but then you try and use the pointer

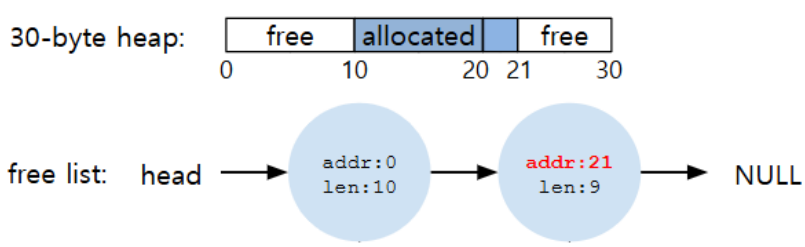


- double free, should be fine if you set the pointer to null after freeing; leads to undefined behavior
-

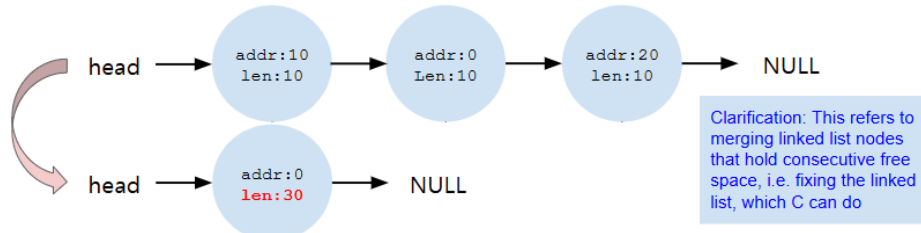
- C is pass by value, so if u pass pointer into a function, that pointer is copied and modified in the function
- double pointer: memory address to a pointer; when you pass this through a function, mem address is copied and you can dereference in the function to malloc or change the value of what the actual pointer is pointing too or what it holds.
- in a 2d matrix on the heap, remember everything is stored in 1 dimension, so to get like M[i][j], you do M[i * num_of_cols + j], num of columns because that how many are in each row, so you jump ahead by i depending on the ith row.
- pointers are a byte address, so ptr++ is like increasing address by whatever the size is. so it pointer is an int, it would jump ahead by 4 bytes.
- 1 byte per character

Free space management

- in C, basically can't do heap compaction. each pointer is allocated through malloc. if you change where that block resides, the code has no way to know where the block was moved to.
- 2 types of memory in heap: allocated (called malloc, even if nothing is inside) or free(freed, nothing currently is bound to it)
    - `MemTotal: Total usable ram (i.e. physical ram minus a few reserved bits and the kernel binary code)`
    - `MemFree: The sum of LowFree+HighFree`
    - `+MemAvailable: An estimate of how much memory is available for starting new`
    - `+ applications, without swapping. Calculated from MemFree,`
    - `+ SReclaimable, the size of the file LRU lists, and the low`
    - `+ watermarks in each zone.`
    - `+ The estimate takes into account that the system needs some`
    - `+ page cache to function well, and that not all reclaimable`
    - `+ slab will be reclaimable, due to items being in use. The`
    - `+ impact of those factors will vary from system to system.`
    - `Buffers: Relatively temporary storage for raw disk blocks`
    - `shouldn't get tremendously large (20MB or so)`
    - `Cached: in-memory cache for files read from the disk`
- basically, there is something called splitting where you find a free chunk of memory to satisfy the memory request and split it into two
    - this is used when the memory allocation is smaller htan the size of free chunks. meaning we split the free chunks into two
    - the list of free blocks is kept as a linked list btw



- if the size you are requesting is bigger than any free chunk size, we do compaction/coalescing
    - you can merge nearby chunks (as in they are right next to each other; contiguous!



- malloc stores extra information in a region of the block allocated to memory. that way, you can do free(void *ptr) and then it can read the first whatever to know how big the block is
    - getting the header:

```
void free(void *ptr) {
    header_t *hptr = (void *)ptr - sizeof(header_t);
}
```

    - header can contain the size as well as a magic number to check for integrity
- free list: totally different than the header shit above; a data structure to keep track of the free blocks; uses linked lists and nodes with the following struct:

```
typedef struct __node_t {
    int size;
    struct __node_t *next;
} nodet_t;
```

- from start, heap has one free chunk, use mmap t o make it
    - `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);`
- use mmap because to reserve heap space and also to create boundary of the heap; making sure it is anonymous and
- then switch back to malloc for allocating memory
    - if you say malloc(100), usually you are allocating 108 bytes, 100 for request, 4 for size, and 4 for the magic (integrity) number
    - will find find first chunk large enough for request, split into two and the shrink the size of free chunk to desired one (this is if the bytes are smaller)
- then, when you free, simply add that pointer to the linkedlist; add it to the head
- this can lead to external fragmentation, when small blocks of free memory are scattered around in the heap
    - scattered, non continuous; there may be enough free memory, but not contiguous to store the block fo data; worse case is when block of wasted memory between every two allocated blocks
    - can also be internal; where you are paging or allocating memory but the process doesn't even need all of the allocation you said you will use, so it will just be wasted
- basic strategies for fragmentation
    - best fit (O(n), traverse entire linked list, find the chunks that are the closest in

size to the amount you are trying to allocate
    - worst fit (O(n)), traverse list to find the largest, then take whatever is left and put back in the free list.
    - first fit(O(n)): first chunk that is big enough for request
    - next fit(O(n)): first chunk, but starting from wherever the free pointer is at, not the start
    - all of these actually suffer from external fragmentation; many will have left over blocks basically and all of them will have a case of that happening
- heap usually calls for more memory and stuff from the OS when it doesn't have enough; calls sbrk() usually not done by the developers though. no mmap() as it doesn't like add to the heap, it creates like a subheap
    - int brk(void *addr);
      brk() sets the end of the data segment to the value specified by addr, when that value is reasonable, the system has enough memory, and the process does not exceed its maximum data size.
      On success, brk() returns zero. On error, -1 is returned, and errno is set to ENOMEM.
      void *sbrk(intptr_t increment);
      sbrk() increments the program's data space by increment bytes. Calling sbrk() with an increment of 0 can be used to find the current location of the program break.
      On success, sbrk() returns the previous program break. (If the break was increased, then this value is a pointer to the start of the newly allocated memory). On error, (void *) -1 is returned, and errno is set to ENOMEM
- mmap awesomeness
- `node_t *head = mmap(NULL, 4096 PROT_READ|PROT_WRITE, MAP_ANON|MAP_PRIVATE, -1, 0);`
- `head->size = 4096 - sizeof(node_t);`
- `head->next = NULL;`
- what this is doing is basically allocating memory tha tis 4096 bytes in the ram initialize to all 0s, using a FD of -1 ot indicate that yes i want 4096 bytes of memory in ram that is private to me. you can change to map_shared to share with a subprocess
- ways to compact
    - you can do it in the virtual memory like java, when free blocks aren't contiguous and just make more room since you can move objects around and can still track the object
    - you can also optimize the physical memory for spatial locality by paging, TLB, or disk swapping, moving the free spaces towards the end; spatial meaning allthe free blocks are right next ot each other and thus given a cache it will all be in the cache and no need to keep fetching it over and over again
    - compaction: to solve external frag